

Learning Which Item to Evict Inside an Adaptive Replacement Cache

Yash Laxman, R Ajay Kumar, Kritharth B Shetty, and Pooja J Naik

Department of Computer Science and Engineering,
Sahyadri College of Engineering & Management, Mangaluru, India

Abstract. Every cache has to decide which item to remove when it runs out of space. Classic rules such as LRU and LFU each work well on some workloads and poorly on others. ARC improves on them by balancing recency against frequency, but once it has picked a list to shrink it always removes the oldest item in that list. This chapter proposes TIDE, a design that keeps ARC exactly as it is and hands the final eviction choice to a deep feedforward neural network trained continually on live traffic. The network is the central contribution of this work: a three-hidden-layer architecture that turns twelve recency, frequency and periodicity features into a single reuse-time score, trained end to end by backpropagation with no offline dataset and no reward shaping. It learns from delayed labels that the request stream produces on its own, replayed through an experience buffer in the style of deep reinforcement learning systems, and it may also decide not to cache a new item at all. We describe the network’s architecture and training procedure in detail, explain why it should help on looping and phase-based workloads and why it cannot help on purely random traffic, show how a shadow copy of ARC limits the damage when the network is wrong, and set out a plan for testing these expectations.

Keywords: Deep learning · Neural networks · Cache replacement · Adaptive replacement cache · Online learning · Eviction policy

1 Introduction

A cache keeps a small number of items close at hand so that repeated requests can be served quickly. When the cache is full and a new item arrives, something has to go. The rule that makes this choice is the replacement policy, and it decides the hit rate: the share of requests the cache can answer without going to slower storage. Even a few points of hit rate can matter a great deal, because every miss costs a disk read, a database query or a network trip.

The best possible rule has been known for a long time. Belady showed that a cache should remove the item that will be needed furthest in the future [1]. Real systems cannot see the future, so they guess. LRU guesses that recently used items will be used again soon. LFU guesses that popular items will stay popular. Each guess holds on some workloads and fails on others.

Learned policies try to replace these fixed guesses with a model trained on real traffic [2, 3]. In practice they tend to throw away the heuristic they replace, they add cost to every decision, and they can do much worse than the old rule when the model is wrong. We take a narrower path. ARC [4] already does one thing well: it decides whether the recency side or the frequency side of the cache should shrink. It does a second thing poorly: inside the chosen side it always removes the oldest item. TIDE keeps the first decision and hands the second to a deep neural network.

The network is deliberately the centrepiece of this chapter. It is a three-hidden-layer feedforward architecture, deep rather than shallow, trained continually on live traffic with no offline dataset, no hand-labelled examples and no reward shaping of the kind reinforcement learning needs. This chapter makes four contributions. It frames eviction as ranking a few candidates by how soon they will return. It presents a complete deep learning based design built on that idea, including the network’s architecture, its input representation and its training procedure. It gives simple arguments for how the design should behave, including its cost and how far it can fall behind ARC. Finally, it turns those arguments into predictions and a plan for testing them.

2 Related Work

2.1 Belady’s Rule

Belady’s rule removes the item whose next request is furthest away, and no other demand paging policy can have fewer misses on the same sequence [1]. It is used as a yardstick. A common way to report results is a normalised hit rate, where 0 means no better than LRU and 1 means as good as Belady’s rule. When requests are drawn at random from a fixed set of popularities, the best rule that does not know the future is simply to keep the most popular items [5].

2.2 Classic Policies

Table 1 lists the classic policies that matter most here. Each one relies on a single idea about the workload, and each has a case where that idea breaks.

2.3 How ARC Works

ARC splits the cache into two lists. T_1 holds items seen once recently and T_2 holds items seen at least twice. It also remembers the keys of recently removed items in two ghost lists, B_1 and B_2 , without keeping their data. If a request hits a ghost list, ARC takes it as a sign that it shrank the wrong list, and it moves a target size p in the other direction. When space is needed, ARC removes an item from T_1 if T_1 is larger than p and from T_2 otherwise.

This choice of list is ARC’s strength. Its weakness is what happens next: it always removes the least recently used item of that list. On a loop that is slightly longer than the cache, the oldest item is exactly the one that will be needed next, so ARC gets no hits at all.

Table 1. Classic replacement policies.

Policy	Main signal	Where it struggles
LRU	Time since last use	Scans and loops longer than the cache
LFU	Number of uses	Holds on to items that are no longer popular
2Q [6]	Trial queue, then LRU	Queue sizes are fixed in advance
LIRS [7]	Gap between uses	Extra bookkeeping and tuning
ARC [4]	Recency and frequency lists	Always removes the oldest item of a list
S3-FIFO [8]	Small and main FIFO queues	No view of when an item returns
SIEVE [9]	FIFO with a visited bit	Only a yes or no view of reuse

2.4 Learned Policies

Several groups have trained models to make eviction decisions (Table 2). Hawkeye learns from Belady’s rule replayed on past requests [10]. LRB trains a model online for content delivery caches [2]. Parrot imitates Belady’s rule with a model trained offline [3]. CACHEUS mixes the advice of two heuristics [11]. HALP is the closest to our idea: a heuristic suggests candidates and a learned model picks one [12]. None of them uses ARC’s choice of list as a starting point.

Table 2. Learned replacement policies.

Policy	Learns from	Role of a heuristic
Hawkeye	Belady’s rule on past requests	None
LRB	Relaxed Belady rule, online	None
Parrot	Belady’s rule, offline	None
CACHEUS	Regret on removed items	Mixes two heuristics
HALP	Preference feedback, online	Suggests candidates
TIDE	Time until next request, online	ARC picks the list and is the fallback

3 Problem Formulation

We want eviction choices that are closer to Belady’s rule without giving up what ARC already does well. We also want the extra cost of each decision to stay small and fixed, and we want the design to fall back to ARC when the model is wrong.

The key quantity is the reuse distance d of an item: how many requests from now it will be asked for again. Belady’s rule removes the item with the largest d . We do not need the exact value, only the order of the candidates, so the model

predicts

$$g(d) = \log \left(1 + \frac{d}{c} \right), \quad (1)$$

where c is the cache size. Dividing by c puts caches of different sizes on the same scale, and the logarithm stops a few very large distances from dominating training.

4 Proposed Method

4.1 Overview

TIDE keeps all of ARC: its four lists, its target size and its choice of which list to shrink. The only change is the last step. Instead of removing the oldest item of the chosen list, TIDE scores a few candidates and removes the one it expects to be needed last. Figure 1 shows the parts of the design.

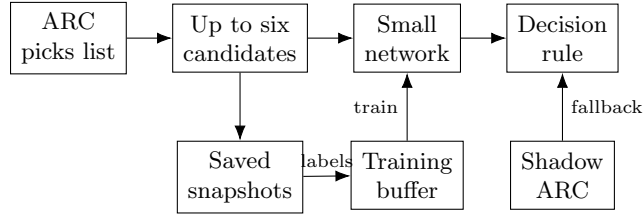


Fig. 1. The parts of TIDE. ARC picks the list, the network ranks a few candidates, and a shadow copy of ARC decides when to hand control back.

4.2 Candidates

For the list ARC has chosen, TIDE looks at up to six items:

- the four oldest items, where the oldest one is ARC’s own choice;
- the newest item, which is often the right one to drop on long loops, because it will not come back until the loop comes round again;
- the item being requested, which lets TIDE serve it without storing it. We call this a bypass. It protects the cache from one-time scans.

The number of candidates never depends on the cache size.

4.3 Features

TIDE keeps a short history for each item it has seen recently, including items that were removed, so it can recognise them when they come back. From this

Table 3. The twelve features computed for each candidate.

No.	Feature	What it captures
1	Time since last request	Recency
2 to 4	Last three gaps between requests	Regular patterns and loops
5	Smoothed average gap	Typical return time
6	Capped request count	Whether the item is new or repeat
7 to 9	Three fading counters	Popularity over short, medium and long spans
10	Total request count	Long term popularity
11	Is this the requested item?	Marks the bypass option
12	Is it in the frequency list?	ARC’s own view of the item

history it builds the twelve numbers in Table 3. Time values are scaled by the cache size in the same way as the target. None of the features look at the content of an item, so the design works with any kind of key.

There is no feature for trend. An item that is gaining popularity already shows shorter gaps and a short term counter that is higher than its long term one, and the network can learn to read that.

4.4 Neural Network Architecture

At the centre of TIDE is a deep feedforward neural network that turns the twelve input features of Sect. 4.3 into a single reuse-time score. Rather than a single hidden layer, the network stacks three hidden layers of decreasing width, so that early layers can combine raw recency and frequency signals into intermediate representations and later layers can sharpen those representations into a final ranking score. This layered structure is what lets the network pick up on combinations of signals, such as a short recent gap together with a rising short-term counter, that no single input feature captures on its own.

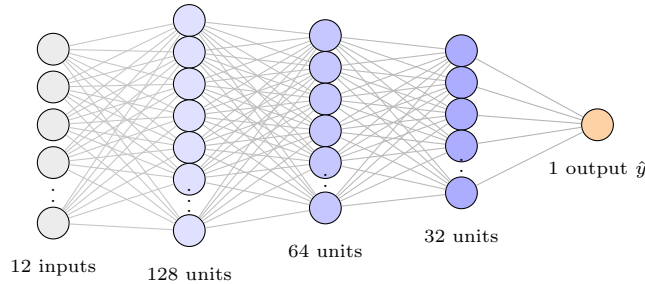


Fig. 2. The deep scoring network: three hidden layers with ReLU activations narrow twelve raw features down to one reuse-time score.

Input representation. Each candidate is represented by the twelve-dimensional feature vector $x \in \mathbb{R}^{12}$ described in Table 3. This plays the same role that an embedding layer plays in larger deep learning systems: it turns raw, irregularly scaled measurements (ages, gaps, counts) into a fixed-size numerical vector that a network layer can consume directly.

Layer stack. The network computes

$$h_1 = \text{ReLU}(W_1 x + b_1), \quad W_1 \in \mathbb{R}^{128 \times 12}, \quad (2)$$

$$h_2 = \text{ReLU}(W_2 h_1 + b_2), \quad W_2 \in \mathbb{R}^{64 \times 128}, \quad (3)$$

$$h_3 = \text{ReLU}(W_3 h_2 + b_3), \quad W_3 \in \mathbb{R}^{32 \times 64}, \quad (4)$$

$$\hat{y} = w_4^\top h_3 + b_4, \quad w_4 \in \mathbb{R}^{32}, \quad (5)$$

with rectified linear activations between layers and a linear output head, giving roughly 12,000 trainable parameters across the four weight matrices. Figure 2 shows the resulting architecture. Each hidden layer halves the width of the one before it, a common design pattern in deep networks that keeps early layers wide enough to preserve information from the raw features while letting later layers specialise toward the regression target.

Training objective. The network is trained end to end by backpropagation to minimise the mean squared error between $\hat{y}$ and the transformed reuse-time target $g(d)$ of Eq. (1). Because the true label for a candidate is only known once it is requested again or its window expires (Sect. 4.6), every training example is a form of delayed, weakly supervised signal, closer to the censored-regression problems studied in survival analysis than to a standard supervised dataset with labels available up front. The network nonetheless learns entirely through gradient descent on this loss, with no reward shaping and no trial-and-error exploration of the kind used in reinforcement learning.

The network is deliberately kept small in absolute terms, even though it is deep in structure: it is evaluated on at most six candidates per decision, so a wide, heavily over-parameterised network would add latency without adding useful representational power. The three-layer design is chosen to be the smallest depth at which the network can combine recency, frequency and periodicity signals nonlinearly, while still running well within a microsecond-scale latency budget.

4.5 Decision Rule

TIDE removes the candidate with the highest score, with one condition: any candidate other than ARC’s own choice has to win by a margin. For the other old items the margin is 0.1, which means the model must expect the item to return roughly 10% later than ARC’s choice. For the newest item and the bypass option the margin is 0.2, or roughly 22% later, because these moves are riskier. When the model is unsure, ARC’s choice stands.

4.6 Online Training

The network is trained continually, in an online, continual-learning setting rather than on a fixed offline dataset. Each time a candidate is scored, its feature vector is saved. When that item is requested again, the saved features get their true label. If the item has not come back after $W = 8c$ requests, it is labelled as “far away” instead. Labelled examples go into a replay buffer of 32,768 rows, the same kind of experience-replay mechanism used to stabilise training in deep reinforcement learning systems, and after every 32 new labels the network takes one stochastic gradient step on a random minibatch of 256 rows using the Adam optimiser [13]. Sampling minibatches at random from the buffer, rather than training on labels in the order they arrive, breaks the correlation between consecutive requests and keeps the gradient estimates stable. The network is not used to make decisions at all until at least 2,048 labels have been collected, so every early decision is ARC’s and the network only takes over once it has seen enough examples to have learned a meaningful mapping from features to reuse time.

4.7 Fallback

A second copy of ARC, the shadow, sees the same requests and keeps only keys. TIDE counts its misses and the shadow’s misses over windows of a few thousand requests. If TIDE misses clearly more in a window, it stops using the model and lets ARC decide for a while. The pause starts at two windows and doubles after each repeated failure, up to 32 windows. Training carries on during the pause. Table 4 lists all settings. They are chosen before any testing and are the same for every workload.

Table 4. Design settings, fixed in advance for all workloads.

Setting	Value	Reason
Candidates per decision	Up to 6	Keeps cost fixed
Label window W	$8c$	Covers several cache turnovers
Hidden layers	128, 64, 32	About 12,000 parameters in total
Training buffer	32,768 rows	Old patterns fade out
Training step	Every 32 labels	Limits training time
Batch size	256	Stable updates
Margins	0.1 and 0.2	Model must be clearly better
Start after	2,048 labels	No untrained decisions
Fallback window	$\max(2000, 4c)$	Enough requests to compare
Fallback tolerance	1% of the window	Small losses are ignored

5 Theoretical Analysis

This section gives short arguments for how the design should behave. We assume that the cache does not change which items users request.

5.1 Labels Come for Free

Every scored candidate gets exactly one label within W requests. The label depends only on when the item is requested again, not on what the cache did. This is the main difference from reinforcement learning. The cache never has to try bad moves to learn, and it never has to work out which of its past actions led to a later miss.

5.2 Cost Stays Fixed

A hit does not use the network at all. A miss runs a forward pass through the three-layer network for at most six candidates, which is about 71,000 multiply and add operations in total, still small enough to execute in a few microseconds on ordinary hardware. The replay buffer takes about 1.7 MB no matter how big the cache is, and the network and optimiser state together take well under a megabyte. Only the per item history grows with the cache, in the same way that ARC's ghost lists do. The cost of a decision therefore does not grow as the cache gets bigger, regardless of how deep the network is, since its depth is fixed in advance and never scales with cache size.

5.3 Loops Longer Than the Cache

Take a cache of 16 items and a workload that requests keys 1 to 17 in order, over and over. LRU always removes the item that is needed next, so it never hits. FIFO does the same, and so does ARC. Belady's rule misses once every 16 requests, which gives a hit rate of 93.75%. A policy that keeps 16 keys fixed and never stores the 17th does even better: it misses once per round of 17 requests, for 94.1%. No policy can beat that, because at least one of the 17 keys must miss in every round.

TIDE has both of the moves needed here. It can remove the newest item, which is what Belady's rule does, and it can skip storing the requested item, which is what the bypass policy does. ARC can do neither.

5.4 When There Is Nothing to Learn

If every request is picked at random from N equally likely items, the chance of a hit is c/N for any policy that keeps the cache full, because the next request has nothing to do with what is in the cache. TIDE cannot beat ARC here, and it should not lose either. When some items are more popular than others but requests are still random, the best possible policy keeps the most popular items [5]. TIDE can only gain the difference between ARC and that best policy, which is usually small.

5.5 How Far It Can Fall Behind ARC

The fallback limits how much worse than ARC TIDE can be over a long run. Suppose the model keeps failing. After a few failures, each window in which the model is used is followed by 32 windows in which ARC decides, so at most about one window in 33 is lost. That caps the long run loss at about 3% of ARC’s hits. A workload that switches back and forth between fooling the model and suiting it perfectly could push the loss up to about a quarter of ARC’s hits, because a single good window resets the pause. Requiring several good windows in a row before trusting the model again would close this gap. Both figures assume that once ARC is back in control, the two caches quickly end up holding similar items.

6 Discussion

Table 5 sums up what the analysis suggests for different kinds of workload.

Table 5. Expected gain of TIDE over ARC by workload type.

Workload	What helps	Expected gain	Why
Uniform random	Nothing	None	No pattern to learn
Skewed random	Popularity counters	Small	Best rule is close to ARC
Loops	Newest item removal, bypass	Large	ARC gets no hits
Scans	Bypass	Moderate	Scan items are skipped
Phase changes	Relearning, fallback	Moderate	Losses are capped
Slow popularity drift	Gap and counter features	Small	Changes are gradual
Strong locality	Nothing	Near zero	LRU is already close to best

The gain should depend on how much structure the workload has. With no pattern there is nothing to learn, and with very strong locality LRU is already close to the best possible. The largest gains should come where recency is misleading, as in loops and scans. For skewed random traffic, the gain should be largest at moderate skew, where recency and popularity disagree most often. For loops, the gain should be largest when the loop is just a little longer than the cache.

At the start of every workload TIDE behaves exactly like ARC, because the model waits for enough labels. After a sudden change in the workload, the model

may make poor choices until the buffer fills with new examples, and the fallback limits the cost of that period. In a prototype written in Python, we expect a decision to take tens of microseconds, which is slower than native ARC but does not grow with the cache.

7 Evaluation Methodology

The expectations above can be tested. Table 6 states them as hypotheses along with the result that would show each one to be wrong, and Table 7 describes the planned setup. All settings are fixed before any test is run.

Table 6. Hypotheses to test.

	Hypothesis	Wrong if
H1	TIDE has the best average normalised hit rate	Any baseline matches or beats it
H2	On loops, TIDE gets close to the bypass rate while ARC stays near zero	TIDE stays close to ARC
H3	On uniform random traffic, TIDE and ARC tie	The gap is larger than run-to-run noise
H4	TIDE never falls far behind ARC	Any case loses more than about one point
H5	Decision time does not grow with cache size	Median decision time rises with the cache

Table 7. Planned test setup.

Item	Plan
Workloads	Uniform and skewed random traffic, loops, scans, bursty and periodic traffic, drifting popularity and sudden phase changes
Cache sizes	0.1%, 1% and 10% of the distinct keys in each workload
Baselines	LRU, LFU, ARC, S3-FIFO and SIEVE
Reference points	Belady’s rule, and TIDE given the true reuse distances
Measures	Normalised hit rate, cases won, worst case against ARC, decision time and training time
Repeats	At least three runs with different seeds

Running TIDE with the true reuse distances in place of the network shows how much of any shortfall comes from the choice of candidates and how much comes from the model.

8 Limitations

The loss bound assumes that TIDE and the shadow quickly hold similar items once ARC is back in control, which is only roughly true. The design also assumes that every item has the same size. Items that return after more than W requests all get the same “far away” label, so the model cannot tell them apart. The candidates the model sees depend on its own earlier choices, so what it trains on may drift from what it is asked to judge. Finally, all settings are fixed in advance, and the design has no way to tune them for a given workload.

9 Conclusion

TIDE keeps ARC’s choice of which list to shrink and hands the choice of which item to remove from it to a deep neural network, with the option of not caching a new item at all. The network is a three-hidden-layer architecture trained end to end by backpropagation, and because its training labels come from the request stream itself, replayed through an experience buffer, it can learn continually while the cache runs, with no offline data and no trial and error. The cost of each decision stays fixed regardless of the network’s depth, and a shadow copy of ARC limits how much the network can hurt. We expect clear gains on loops, scans and phase changes, small gains on skewed random traffic, and none on uniform random traffic. The evaluation plan in Sect. 7 is designed to check each of these expectations.

References

1. L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
2. Z. Song, D. S. Berger, K. Li, and W. Lloyd. Learning relaxed Belady for content distribution network caching. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 529–544, 2020.
3. E. Z. Liu, M. Hashemi, K. Swersky, P. Ranganathan, and J. Ahn. An imitation learning approach for cache replacement. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 6237–6247, 2020.
4. N. Megiddo and D. S. Modha. ARC: A self-tuning, low overhead replacement cache. In *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST)*, pages 115–130, 2003.
5. A. V. Aho, P. J. Denning, and J. D. Ullman. Principles of optimal page replacement. *Journal of the ACM*, 18(1):80–93, 1971.
6. T. Johnson and D. Shasha. 2Q: A low overhead high performance buffer management replacement algorithm. In *Proceedings of the 20th International Conference on Very Large Data Bases*, pages 439–450, 1994.
7. S. Jiang and X. Zhang. LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance. In *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, pages 31–42, 2002.

8. J. Yang, Y. Zhang, Z. Qiu, Y. Yue, and R. Vinayak. FIFO queues are all you need for cache eviction. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 130–149, 2023.
9. Y. Zhang, J. Yang, Y. Yue, Y. Vigfusson, and K. V. Rashmi. SIEVE is simpler than LRU: An efficient turn-key eviction algorithm for web caches. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024.
10. A. Jain and C. Lin. Back to the future: Leveraging Belady’s algorithm for improved cache replacement. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, pages 78–89, 2016.
11. L. V. Rodriguez, F. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, and G. Narasimhan. Learning cache replacement with CACHEUS. In *Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST)*, pages 341–354, 2021.
12. Z. Song, K. Chen, N. Sarda, D. Altınbüken, E. Brevdo, J. Coleman, X. Ju, P. Jurczyk, R. Schooler, and R. Gummadi. HALP: Heuristic aided learned preference eviction policy for YouTube content delivery network. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 1149–1163, 2023.
13. D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.